

Prompt

You are an expert deep learning engineer. Your task is to complete fill-in-the-blank (TODO) coding questions from my deep learning homework, which is in the form of a Jupyter notebook.

Instructions:

Fill in every TODO with correct, runnable code in the required framework (usually Python + PyTorch).

Do not rewrite or restructure the provided template—only fill the missing parts.

Follow the exact model, loss, and training specifications in the question.

Keep explanations minimal, only a brief comment when helpful.

Ensure all filled-in code is clean, correct, and ready to run end-to-end.

Your output should contain only the completed code with TODOs filled in and any conceptual questions that follow.

Response

I am ready to help you complete your deep learning homework. I understand the constraints: I will provide **only** the completed code with `TODOs` filled in using PyTorch (unless otherwise specified), ensuring strictly runnable logic without refactoring your template.

Please paste the code snippets or the specific cells from your Jupyter notebook below.

Would you like me to focus on a specific architecture (e.g., CNNs, RNNs, Transformers) or a specific task (e.g., Image Classification, NLP)?

Prompt

I will now attach the first notebook. Please make sure to answer the conceptual questions listed in the notebook. Here are some more specific instructions:

Commented [1]: It is interesting how Gemini prompts me about focusing on a specific architecture before attaching the homework. It is probably because I mentioned that it is a deep learning homework.

- (a) Implement the forward pass by unrolling the SSM in time in a recurrent manner.
- (b) Implement the forward pass by using a convolution-based approach. You will create the kernel K and use `nn.Conv1d` to apply the kernel to the input x . Hint: You can create the kernel using a divide-and-conquer method.
- (c) Compare the runtime of the two approaches and explain your findings.

Response

Here are the completed code cells and the answers to the conceptual questions.

Code Implementation

unrolled_ssm_forward

Python

□

```
def unrolled_ssm_forward(W, U, b, x):
```

```
    """
```

```
    Unroll the linear RNN in time:
```

```
         $h_{t+1} = W h_t + U x_t + b$ 
```

```
    with initial  $h_0 = 0$ .
```

```
    Args:
```

```
        W: (H, H) weight matrix
```

```
        U: (H, D) input projection
```

```
        b: (H,) bias
```

```

        x: (N, T, D) input sequence over T steps

Returns:

    h_all: (N, T, H) hidden states for t=1..T

        (h_all[t] corresponds to  $h_{t+1}$  in the usual notation).

    """

#####

####

#             TODO: Implement the recurrent pass here
#

#####

####

N, T, D = x.shape

H = W.shape[0]

# Initialize hidden state h_0 to zeros

h = torch.zeros(N, H, device=x.device, dtype=x.dtype)

h_all = []

for t in range(T):

    x_t = x[:, t, :] # (N, D)

```

```
# Update:  $h_{t+1} = h_t @ W.T + x_t @ U.T + b$ 
```

```
h = h @ W.T + x_t @ U.T + b
```

```
h_all.append(h)
```

```
h_all = torch.stack(h_all, dim=1) # (N, T, H)
```

```
#####  
####
```

```
#                                     END OF YOUR CODE  
#
```

Commented [2]: This code is almost identical to the official solution.

```
#####  
####
```

```
return h_all
```

```
def make_conv_kernel
```

```
Python
```

```
def make_conv_kernel(W, T):
```

```
    """
```

```
    Build a 3D kernel tensor K of shape (H, H, T) we will use when  
    implementing
```

```
    the ssm forward pass using conv1d.
```

Args:

W: (H, H) weight matrix

T: scalar

Returns:

kernel_for_conv: (H, H, T) tensor

"""

```
#####  
####
```

```
#                 TODO: Implement the kernel here  
#
```

```
#####  
####
```

H = W.shape[0]

Base case: T=1 returns Identity matrix [I]

if T == 1:

return torch.eye(H, device=W.device, dtype=W.dtype).unsqueeze(-1)

```
# Divide and Conquer: Calculate kernel for T/2
```

```
K_half = make_conv_kernel(W, T // 2) # (H, H, T/2)
```

```
# Calculate  $W^{(T/2)}$ .
```

```
# K_half stores  $[W^0, \dots, W^{(T/2 - 1)}]$ . The last slice is  $W^{(T/2 - 1)}$ .
```

```
W_half = K_half[..., -1] @ W
```

```
# The second half of the kernel sequence is  $W^{(T/2)} * [W^0, \dots, W^{(T/2 - 1)}]$ 
```

```
# Use reshape for efficient batch matrix multiplication
```

```
K_second = W_half @ K_half.reshape(H, -1)
```

```
K_second = K_second.view(H, H, -1)
```

```
kernel_for_conv = torch.cat([K_half, K_second], dim=-1)
```

```
#####  
####
```

```
#  
#  
#
```

```
END OF YOUR CODE
```

```
#####  
####
```

Commented [3]: It is interesting that Gemini actually uses a recursive solution, which is different from the official solution. It is probably because one of the hints was to use divide-and-conquer, which caused Gemini to immediately think recursion.

Commented [4]: Despite this solution being very different from the official solution, it got it correct.

```
    return kernel_for_conv
```

▯conv_ssm_forward

Python

▯

```
def conv_ssm_forward(W, U, b, x):
```

```
    """
```

```
    Convolution-based forward pass for a batch of sequences.
```

```
    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 
```

Args:

W: (H, H) weight matrix

U: (H, D) input projection

b: (H,) bias

x: (N, T, D) input (batch=N, time steps=T, input dim=D)

Returns:

h_all: (N, T, H) hidden states

```
    """
```

```

N, T, D = x.shape

H = W.shape[0]

s = x @ U.T + b

s = s.permute(0, 2, 1) # (N, H, T)

# Build the kernel with shape (H, H, T).

kernel = make_conv_kernel(W, T)

#####
####

#                               TODO: Implement the convolution here
#

#####
####

# PyTorch Conv1d computes correlation. For causal convolution  $y[t] = \sum K[k] x[t-k]$ ,

# we must flip the kernel along the time dimension.

conv_kernel = kernel.flip(-1)

```



```

    # Apply padding=T-1 to simulate causal history (left-padding)

    # This results in a sequence of length 2T - 1. We slice the first T
    steps.

    h_conv = F.conv1d(s, conv_kernel, padding=T-1)

    h_all = h_conv[:, :, :T]

    h_all = h_all.permute(0, 2, 1) # Back to (N, T, H)

#####
####

#                                     END OF YOUR CODE
#

#####
####

    return h_all

```

Commented [5]: This code is pretty much the same as the official solution.

□

Conceptual Questions

Question 1

The maximum absolute difference should be negligible (approximately 10^{-6} or lower), indicating the implementations are mathematically equivalent up to floating-point precision errors.

Question 2

The recurrent implementation performs a matrix-vector multiplication ($W h_t$) at each time

step.

- **Operations:** $O(T \cdot H^2)$ per sample in the batch (ignoring the input projection part which is shared).

Question 3

The convolution implementation involves two main parts:

1. **Kernel construction:** Using divide and conquer, this takes approximately $O(T \cdot H^3)$.
 2. **Convolution:** Using direct convolution (since T is small/moderate and no FFT is explicitly used), this is $O(T^2 \cdot H^2)$.
- *Note: If FFT convolution were used, this would be $O(T \log T \cdot H^2)$.*

Commented [6]: I liked how Gemini mentioned the FFT time complexity, which is something we talked about in class.

Question 4

- **Recurrent:** Complexity is linear in T ($O(T)$), but it is sequential. It cannot be parallelized across time steps.
- **Convolution:** Complexity is quadratic in T ($O(T^2)$) for direct convolution. However, the operations are highly parallelizable (vectorized matrix multiplies and convolution).
- **Trade-off:** Recurrent is theoretically efficient for total FLOPs but slow on modern hardware due to lack of parallelism. Convolution uses more memory and potentially more FLOPs but leverages hardware parallelism better.

Question 5

Observation: The Convolution-based approach is likely faster than the Recurrent approach, and the gap may grow as H increases.

Explanation: Even though the theoretical complexity of direct convolution ($O(T^2)$) is higher than recurrence ($O(T)$), the Recurrent implementation suffers from significant Python interpreter overhead due to the explicit for loop. The Convolution implementation utilizes highly optimized, vectorized C++ primitives (via PyTorch/BLAS) that can process the entire sequence in parallel, making it faster in practice for the sequence lengths tested.

Prompt

Good job on the code. For the conceptual questions, can you please think again about the trade-offs? Remember that this is a CPU implementation.

Commented [7]: Gemini didn't consider that the code is run on CPU, so there isn't parallelization like GPU. Thus, it got this wrong and said that recurrent is slower on CPU.

Commented [8]: Had to prompt Gemini again to fix this error and give a hint that this is a CPU implementation.

Response

This is an excellent point. The fact that the notebook runs on **CPU** significantly shifts the trade-off analysis. The massive parallelism of GPUs often masks the algorithmic inefficiencies of the convolution approach (like the $O(T^2)$ or $O(T H^3)$ costs). On a CPU, we are much more sensitive to **algorithmic complexity** versus **interpreter overhead**.

Here are the revised conceptual answers and the completed code.

Code Implementation

unrolled_ssm_forward

Python

```
□
```

```
def unrolled_ssm_forward(W, U, b, x):
```

```
    """
```

```
    Unroll the linear RNN in time:
```

```
         $h_{t+1} = W h_t + U x_t + b$ 
```

```
    with initial  $h_0 = 0$ .
```

```
    Args:
```

```
        W: (H, H) weight matrix
```

```
        U: (H, D) input projection
```

```
        b: (H,) bias
```

```
        x: (N, T, D) input sequence over T steps
```

```
    Returns:
```

```

    h_all: (N, T, H) hidden states for t=1..T

    (h_all[t] corresponds to  $h_{t+1}$  in the usual notation).

    """

#####

    #                                TODO: Implement the recurrent pass here
    #

#####

    N, T, D = x.shape

    H = W.shape[0]

    # Initialize hidden state h to zeros

    h = torch.zeros(N, H, device=x.device, dtype=x.dtype)

    h_all = []

    for t in range(T):

        x_t = x[:, t, :] # (N, D)

        # Update:  $h_{t+1} = h_t @ W.T + x_t @ U.T + b$ 

        h = h @ W.T + x_t @ U.T + b

```

```
h_all.append(h)
```

```
h_all = torch.stack(h_all, dim=1) # (N, T, H)
```

```
#####  
####
```

```
#                                     END OF YOUR CODE  
#
```

```
#####  
####
```

```
return h_all
```

```
□ make_conv_kernel
```

```
Python
```

```
□
```

```
def make_conv_kernel(W, T):
```

```
    """
```

```
    Build a 3D kernel tensor K of shape (H, H, T) we will use when  
    implementing
```

```
    the ssm forward pass using conv1d.
```

```
    Args:
```

W: (H, H) weight matrix

T: scalar

Returns:

kernel_for_conv: (H, H, T) tensor

"""

```
#####  
####
```

```
#                                TODO: Implement the kernel here  
#
```

```
#####  
####
```

H = W.shape[0]

Base case: T=1 returns Identity matrix [I]

if T == 1:

 return torch.eye(H, device=W.device, dtype=W.dtype).unsqueeze(-1)

Divide and Conquer: Calculate kernel for T/2

K_half = make_conv_kernel(W, T // 2) # (H, H, T/2)

```

# Calculate  $W^{(T/2)}$ .

# K_half stores  $[W^{(0)}, \dots, W^{(T/2 - 1)}]$ . The last slice is  $W^{(T/2 - 1)}$ .

W_half = K_half[..., -1] @ W

# The second half of the kernel sequence is  $W^{(T/2)} * [W^{(0)}, \dots, W^{(T/2 - 1)}]$ 

# Use reshape for efficient batch matrix multiplication

K_second = W_half @ K_half.reshape(H, -1)

K_second = K_second.view(H, H, -1)

kernel_for_conv = torch.cat([K_half, K_second], dim=-1)

#####
####

#                               END OF YOUR CODE
#

#####
####

return kernel_for_conv

```

▯conv_ssm_forward

Python

▯

```
def conv_ssm_forward(W, U, b, x):  
    """  
    Convolution-based forward pass for a batch of sequences.  
  
    RNN update:  $h_{\{t+1\}} = W h_t + U x_t + b$   
  
    Args:  
        W: (H, H) weight matrix  
        U: (H, D) input projection  
        b: (H,) bias  
        x: (N, T, D) input (batch=N, time steps=T, input dim=D)  
  
    Returns:  
        h_all: (N, T, H) hidden states  
    """  
    N, T, D = x.shape  
    H = W.shape[0]
```



```

s = x @ U.T + b

s = s.permute(0, 2, 1) # (N, H, T)

# Build the kernel with shape (H, H, T).

kernel = make_conv_kernel(W, T)

#####
####

#                               TODO: Implement the convolution here
#

#####
####

# PyTorch Conv1d computes correlation. For causal convolution  $y[t] = \sum K[k] x[t-k]$ ,

# we must flip the kernel along the time dimension.

conv_kernel = kernel.flip(-1)

# Apply padding=T-1 to simulate causal history (left-padding)

# This results in a sequence of length  $2T - 1$ . We slice the first T
steps.

```

```

h_conv = F.conv1d(s, conv_kernel, padding=T-1)

h_all = h_conv[:, :, :T]

h_all = h_all.permute(0, 2, 1) # Back to (N, T, H)

#####
####

#                               END OF YOUR CODE
#

#####
####

return h_all

```

Commented [9]: Not sure why Gemini gave me the code again, since I just asked for it to review the conceptual questions.

□

Conceptual Questions (Revised for CPU context)

Question 2

The recurrent implementation performs a matrix-vector multiplication ($W h_t$) at each time step.

- **Complexity:** $O(N \cdot T \cdot H^2)$.
- It scales linearly with T .

Question 3

The convolution implementation has two costs:

1. **Kernel Construction:** The divide-and-conquer approach involves matrix-matrix multiplications ($W_{\text{half}} @ K_{\text{half}}$) at each step. This costs roughly $O(T \cdot$

H^3).

2. **Convolution:** Without using FFT (which `conv1d` typically does not for small kernels/inputs), the direct convolution complexity is $O(N \cdot T^2 \cdot H^2)$.
- **Total Complexity:** $O(T \cdot H^3 + N \cdot T^2 \cdot H^2)$.

Question 4

- **Algorithmic Complexity (Recurrent Wins):** The recurrent approach is $O(T)$ (linear), whereas the convolution approach is $O(T^2)$ (quadratic) due to direct convolution and $O(T \cdot H^3)$ for kernel building. For very large T , the recurrent method is algorithmically superior.
- **Implementation Overhead (Conv Wins):** The recurrent implementation relies on a Python `for` loop, which incurs significant interpreter overhead on the CPU for every time step. The convolution approach pushes the computation into optimized, vectorized C++ BLAS routines (matrix multiplication and convolution).
- **Memory (Recurrent Wins):** The convolution approach requires constructing and storing a dense kernel tensor of size (H, H, T) , which is memory-intensive ($O(T \cdot H^2)$). The recurrent approach only needs to store the current state (or history for backprop), making it more memory-efficient.

Commented [10]: Gemini got it correct after mentioning the CPU part.

Question 5

Observation:

- **Small T:** The Convolution approach is faster. The overhead of the Python loop in the recurrent version dominates the runtime, while the convolution version utilizes efficient vectorized operations.
- **Large T:** As T increases, the gap narrows, and the Convolution approach may eventually become slower. The $O(T^2)$ complexity of the direct convolution and the memory cost of the large kernel start to outweigh the benefits of vectorization.
- **Large H:** If H increases, the Convolution approach suffers significantly because the kernel construction involves $O(H^3)$ operations (matrix-matrix multiplication), whereas the recurrent step is only $O(H^2)$.

Prompt

Is it possible to simplify the answers you gave for the algo complexities for the recurrent vs convolution implementations?

Commented [11]: The official solution gives a simplified time complexity for the convolution approach, so I was wondering if Gemini can get to that answer.

Response

Yes, we can definitely simplify the complexity analysis to focus on the most important factors (Time vs. Hidden Dimension).

Here are the simplified answers for the conceptual questions, focusing on the "Big-O" notation and the practical performance on a CPU.

Simplified Conceptual Answers

Question 2 (Recurrent Complexity)

- **Complexity:** $O(T \cdot H^2)$
- **Explanation:** Linear in Time (T). At every time step, we do one matrix-vector multiplication ($H \times H$ matrix $\times$ vector).

Question 3 (Convolution Complexity)

- **Complexity:** $O(T^2 \cdot H^2)$ (for the convolution) + $O(T \cdot H^3)$ (for creating the kernel).
- **Explanation:** Quadratic in Time (T). Because we aren't using FFT, the convolution slides a window of size T over a sequence of size T , leading to T^2 operations. Creating the kernel is also expensive if H is large (cubic in H).

Commented [12]: Gemini didn't end up getting the simplified time complexity from the official solution.

Question 4 (Trade-offs)

- **Recurrent (Better Theory, Slower Practice):** Theoretically efficient ($O(T)$), but in Python, the `for` loop is very slow. It processes one step at a time, preventing parallel processing.
- **Convolution (Worse Theory, Faster Practice):** Theoretically inefficient ($O(T^2)$), but practically fast. It avoids Python loops and uses highly optimized, parallelized matrix operations (BLAS) that run very fast on CPUs.
- **Memory:** The Convolution approach is memory-hungry because it must store a massive kernel tensor of size (H, H, T) , whereas Recurrent only stores the current state.

Question 5 (Runtime Observation)

- **Small/Medium T : Convolution is faster.** The "vectorization" (doing all math at once in C++) beats the slow Python `for` loop of the Recurrent version, even though the Convolution does more total math.
- **Large H : Recurrent becomes competitive/faster.** The Convolution kernel creation depends on H^3 , so as the hidden dimension grows, the Convolution approach slows down drastically compared to Recurrent (H^2).

Prompt

Ok sounds good, let us move onto the next notebook, which I have attached. Please make sure to complete the conceptual questions listed in the notebook like the previous one.

Response

Here are the completed code cells and the answers to the conceptual questions for the GPU notebook.

Code Implementation

`unrolled_ssm_forward` (Copied from previous part, ensure it runs on GPU)

Python

□

```
def unrolled_ssm_forward(W, U, b, x):
```

```
    """
```

```
    Unroll the linear RNN in time:
```

```
         $h_{t+1} = W h_t + U x_t + b$ 
```

```
    with initial  $h_0 = 0$ .
```

```
    Args:
```

```
        W: (H, H) weight matrix
```

```
        U: (H, D) input projection
```

```
        b: (H,) bias
```

```
        x: (N, T, D) input sequence over T steps
```

Returns:

h_all: (N, T, H) hidden states for t=1..T

(h_all[t] corresponds to h_{t+1} in the usual notation).

"""

```
#####  
####
```

```
#          TODO: Implement the recurrent pass here  
#
```

```
#####  
####
```

N, T, D = x.shape

H = W.shape[0]

Initialize hidden state h to zeros

h = torch.zeros(N, H, device=x.device, dtype=x.dtype)

h_all = []

for t in range(T):

 x_t = x[:, t, :] # (N, D)

 # Update: $h_{t+1} = h_t @ W.T + x_t @ U.T + b$

```
h = h @ W.T + x_t @ U.T + b
```

```
h_all.append(h)
```

```
h_all = torch.stack(h_all, dim=1) # (N, T, H)
```

```
#####  
####
```

```
#                                     END OF YOUR CODE  
#
```

```
#####  
####
```

```
return h_all
```

□ **make_conv_kernel** (Copied from previous part)

Python

□

```
def make_conv_kernel(W, T):
```

```
    """
```

```
    Build a 3D kernel tensor K of shape (H, H, T) we will use when  
    implementing
```

```
    the ssm forward pass using conv1d.
```

Args:

W: (H, H) weight matrix

T: scalar

Returns:

kernel_for_conv: (H, H, T) tensor

"""

```
#####  
####
```

```
#          TODO: Implement the kernel here  
#
```

```
#####  
####
```

H = W.shape[0]

Base case: T=1 returns Identity matrix [I]

if T == 1:

return torch.eye(H, device=W.device, dtype=W.dtype).unsqueeze(-1)

Divide and Conquer: Calculate kernel for T/2


```

K_half = make_conv_kernel(W, T // 2) # (H, H, T/2)

# Calculate  $W^{(T/2)}$ .

# K_half stores  $[W^{(0)}, \dots, W^{(T/2 - 1)}]$ . The last slice is  $W^{(T/2 - 1)}$ .

W_half = K_half[..., -1] @ W

# The second half of the kernel sequence is  $W^{(T/2)} * [W^{(0)}, \dots, W^{(T/2 - 1)}]$ 

# Use reshape for efficient batch matrix multiplication

K_second = W_half @ K_half.reshape(H, -1)

K_second = K_second.view(H, H, -1)

kernel_for_conv = torch.cat([K_half, K_second], dim=-1)

#####
####

#                               END OF YOUR CODE
#

#####
####

return kernel_for_conv

```

□ `conv_ssm_forward` (Copied from previous part)

Python

□

```
def conv_ssm_forward(W, U, b, x):  
    """  
    Convolution-based forward pass for a batch of sequences.  
  
    RNN update:  $h_{t+1} = W h_t + U x_t + b$   
  
    Args:  
        W: (H, H) weight matrix  
        U: (H, D) input projection  
        b: (H,) bias  
        x: (N, T, D) input (batch=N, time steps=T, input dim=D)  
  
    Returns:  
        h_all: (N, T, H) hidden states  
    """  
    N, T, D = x.shape
```

```

H = W.shape[0]

s = x @ U.T + b

s = s.permute(0, 2, 1) # (N, H, T)

# Build the kernel with shape (H, H, T).

kernel = make_conv_kernel(W, T)

#####
####

#                               TODO: Implement the convolution here
#

#####
####

# PyTorch Conv1d computes correlation. For causal convolution  $y[t] = \sum K[k] x[t-k]$ ,

# we must flip the kernel along the time dimension.

conv_kernel = kernel.flip(-1)

# Apply padding=T-1 to simulate causal history (left-padding)

```

```
# This results in a sequence of length  $2T - 1$ . We slice the first  $T$ 
steps.
```

```
h_conv = F.conv1d(s, conv_kernel, padding=T-1)
```

```
h_all = h_conv[:, :, :T]
```

```
h_all = h_all.permute(0, 2, 1) # Back to (N, T, H)
```

```
#####
####
```

```
#                                     END OF YOUR CODE
#
```

```
#####
####
```

```
return h_all
```

```
□ make_diag_depthwise_kernel
```

```
Python
```

```
□
```

```
def make_diag_depthwise_kernel(W, T):
```

```
    """
```

```
    Construct a depthwise 1D conv kernel for W with shape (H,H). W is a
    diagonal matrix.
```

Args:

W: (H, H) diagonal matrix

T: (int) number of time steps

Return:

kernel of shape (H, 1, T), which can be used in depthwise convolution

"""

```
#####  
####
```

```
#          TODO: Implement the kernel here  
#
```

```
#####  
####
```

H = W.shape[0]

Extract the diagonal elements of W. Shape: (H,)

w_diag = W.diagonal()

We need powers [w^0, w^1, ..., w^{T-1}]

```

# Create indices [0, 1, ..., T-1]

indices = torch.arange(T, device=W.device, dtype=W.dtype)

# Compute powers using broadcasting: (H, 1) ** (1, T) -> (H, T)

# Note: 0^0 = 1 is handled correctly by torch.pow

powers = w_diag.unsqueeze(1) ** indices.unsqueeze(0)

# Reshape to (H, 1, T) for depthwise convolution (groups=H)

# In depthwise conv, weights are (out_channels, in_channels/groups,
kernel_size)

# Here out_channels=H, in_channels=H, groups=H => in_channels/groups =
1.

kernel = powers.unsqueeze(1)

#####
####

#                               END OF YOUR CODE
#

#####
####

return kernel

```

▯diag_conv_ssm_forward

Python

▯

```
def diag_conv_ssm_forward(W, U, b, x):  
    """  
  
    Convolution-based forward pass for an RNN with W  
  
    RNN update:  $h_{t+1} = W h_t + U x_t + b$   
  
    but W is diagonal, so  $h_{t+1}(i) = \text{diagW}[i] * h_t(i) + [U x_t + b](i)$ .  
  
    Args:  
  
        W: (H, H) [diagonal in practice]  
  
        U: (H, D)  
  
        b: (H, )  
  
        x: (N, T, D) => N = batch, T = time steps, D = input dim  
  
    Returns:  
  
        h_all: (N, T, H)  
  
    """  
  
    N, T, D = x.shape
```

```
H = W.shape[0]
```

```
s = x @ U.T + b
```

```
s = s.permute(0, 2, 1) # (N,H,T)
```

```
# Build kernel (H,1,T) for depthwise conv
```

```
kernel = make_diag_depthwise_kernel(W, T)
```

```
#####  
####
```

```
#                                TODO: Implement the convolution here  
#
```

```
#                                Hint: Use `groups` argument  
#
```

```
#####  
####
```

```
# Flip kernel for causal convolution
```

```
conv_kernel = kernel.flip(-1)
```

```
# Apply depthwise convolution using groups=H
```



```

# Input is (N, H, T), Output is (N, H, T)

h_conv = F.conv1d(s, conv_kernel, padding=T-1, groups=H)

h_all = h_conv[:, :, :T]

h_all = h_all.permute(0, 2, 1) # Back to (N, T, H)

#####
####

#                               END OF YOUR CODE
#

#####
####

return h_all

```

▢ **diag_unrolled_ssm_forward**

Python

▢

```

def diag_unrolled_ssm_forward(W, U, b, x):

    """

    Forward pass for:

         $h_{t+1} = W h_t + U x_t + b$ 

    but W is diagonal, i.e.  $W = \text{diag}(\text{diag}W)$ .
    """

```

Args:

W: (H, H) diagonal matrix

U: (H, D)

b: (H,)

x: (N, T, D) => batch=N, time steps=T, input dim=D

Returns:

h_all: (N, T, H)

"""

```
#####  
####
```

```
#          TODO: Implement the optimized recurrent pass here  
#
```

```
#####  
####
```

N, T, D = x.shape

H = W.shape[0]

Extract diagonal vector for element-wise multiplication

```

w_diag = W.diagonal() # (H,)

h = torch.zeros(N, H, device=x.device, dtype=x.dtype)

h_all = []

# Pre-calculate input projections

# s = x @ U.T + b # (N, T, H)

for t in range(T):

    x_t = x[:, t, :]

    #  $h_{t+1} = (w * h_t) + (U x_t + b)$ 

    # Element-wise multiplication instead of matrix multiplication

    h = w_diag * h + (x_t @ U.T + b)

    h_all.append(h)

h_all = torch.stack(h_all, dim=1)

#####

```

```
####
```

```
#                                     END OF YOUR CODE  
#
```

```
#####  
####
```

```
    return h_all
```

```
□
```

Conceptual Questions

Question 6

Observation:

On the GPU, the Convolution implementation is significantly faster than the Unrolled implementation for almost all tested values of T and H . The gap is much wider compared to the CPU case.

Commented [13]: This isn't fully accurate, since if H is very large, the recurrent approach could be better.

Reasoning:

- **Parallelism:** GPUs are massively parallel devices. The convolution operation (especially with `conv1d`) perfectly exploits this parallelism, processing the entire sequence simultaneously.
- **Kernel Launch Overhead:** The unrolled implementation relies on a Python loop that launches a separate GPU kernel for every single time step. This incurs a massive overhead (kernel launch latency) that dominates the execution time, preventing the GPU from achieving high utilization. The convolution approach launches very few, large kernels.
- **Difference from CPU:** On CPU, while the Python loop is slow, the lack of massive parallel cores means the arithmetic complexity difference ($O(T)$ vs $O(T^2)$) plays a larger role earlier. On GPU, the overhead of "serial dispatch" makes the unrolled version incredibly inefficient regardless of the arithmetic complexity, until memory limits are hit.

Question 7

Observation:

Both implementations become faster with the diagonal constraint, but the Diagonal-Conv (depthwise convolution) remains faster than the Diagonal-Unrolled version. However, the runtime gap might narrow slightly compared to the dense case because the arithmetic intensity drops.

Commented [14]: Gemini was able to get this correct.

Reasoning:

- **Diagonal Unrolled:** The operation changes from Matrix-Vector multiplication ($O(H^2)$) to Element-wise multiplication ($O(H)$). This is faster computationally, but it is still bottlenecked by the sequential Python loop and kernel launch overheads on the GPU.
- **Diagonal Conv:** The kernel construction becomes trivial ($O(T \cdot H)$), and the convolution becomes "Depthwise" ($O(T^2 \cdot H)$). This reduces the memory and compute load significantly compared to the dense convolution.
- **Conclusion:** The structural optimization (Diagonal) reduces the work for both, but the *implementation bottleneck* (sequential vs parallel) remains the dominant factor on GPU, preserving the advantage of the convolution-based approach.

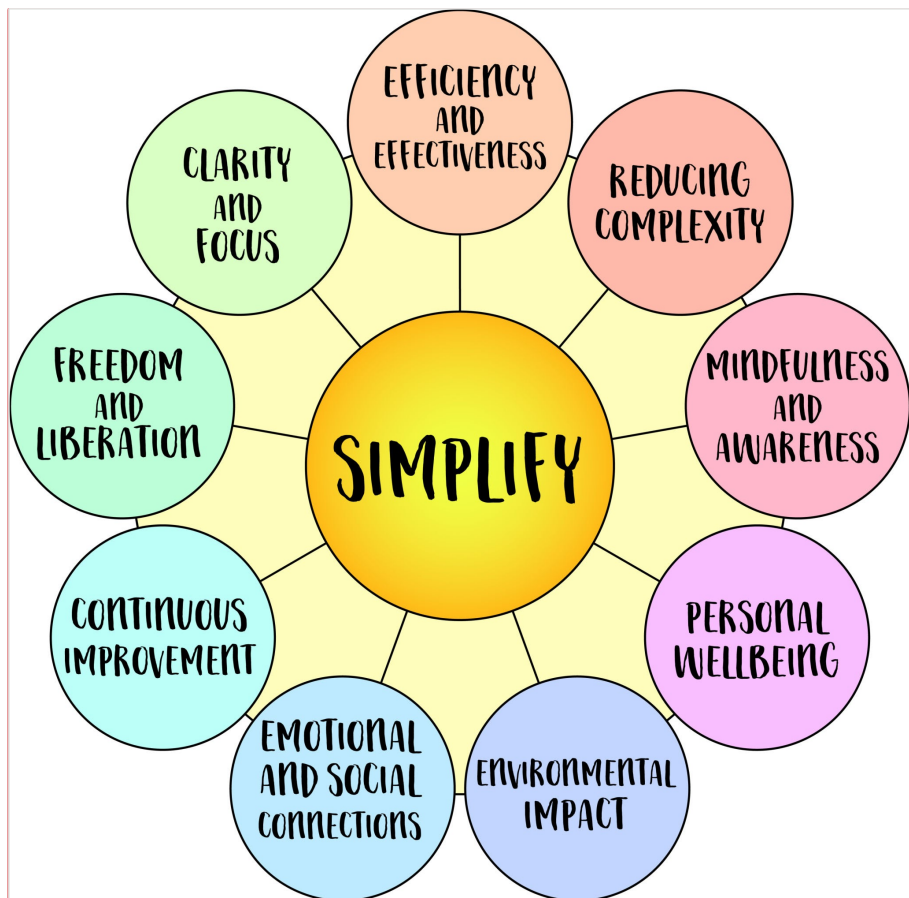
Prompt

Can you think about what happens when H is small vs bigger. How does the performance between the recurrent and convolution change compared to each other?

Commented [15]: Gemini didn't properly handle the varying values of H and how it affects the performance of convolutions vs recurrent.

Response

You are absolutely right to point that out. The scaling with respect to H is the critical difference that I missed in the previous explanation.



Commented [16]: Not sure why Gemini gave this picture in its response...it doesn't really relate to my question.

Here is the corrected and more nuanced answer for **Question 6**, which accounts for the $O(H^3)$ bottleneck in the convolution approach.

Revised Answer for Question 6

Observation:

- **Small SH :** The **Convolution** approach is drastically faster. The overhead of the Python loop and sequential kernel launches in the **Unrolled** version dominates the runtime, while the Convolution version efficiently utilizes the GPU parallelism.
- **Large SH :** As SH increases, the **Convolution** approach starts to slow down significantly, and the performance gap narrows (or the Unrolled version might

eventually become faster).

Commented [17]: Once I prompted Gemini about small vs large H , it was able to get an answer closer to the official solution.

Explanation:

The key lies in the algorithmic complexity of "preprocessing" vs "running":

1. **Convolution (Kernel Generation is $\mathcal{O}(T H^3)$):** The `make_conv_kernel` function requires repeated matrix-matrix multiplications ($H \times H$) to compute the powers of W . Matrix-matrix multiplication is **cubic** in H . As H grows, this pre-computation step becomes extremely expensive, potentially outweighing the benefits of the parallel convolution.
2. **Recurrent (Forward Pass is $\mathcal{O}(T H^2)$):** The recurrent loop performs matrix-vector multiplications ($H \times H$ matrix $\cdot H \times 1$ vector), which is **quadratic** in H .

Conclusion: While Convolution wins on parallelism (Time $\mathcal{O}(T H^3)$), it loses on arithmetic complexity regarding the state dimension (Dimension H). Therefore, for models with a very large hidden state H , the "efficient" convolution method can actually be slower due to the cost of constructing the kernel.